Inhaltsübersicht

1. Einleitung und thematische Annäherung
2. Grundlagen, Grundkonzepte und Architektur von Datenbanksystemen
3. Konzeptioneller (fachlicher) Datenbank-Entwurf und das Entity-Relationship-Modell
4. Das relationale Datenmodell
5. SQL
6. Relationaler Schemaentwurf
7. Datenbank-Transaktionen und Transaktionsverwaltung
8. Physische Datenbank-Organisation
9. Ausblick auf weiterführende DBS-bezogene Themen

Relationaler Schemaentwurf: Worum geht es?

- **Relationales DB-Schema:**
  - **Menge von Relationstypen**
  - **Jeder Relationstyp ist eine „*sinnvolle*“ Gruppierung (semantisch / fachlich) „*zusammengehöriger*“ Attribute**

- **Schema wird erstellt …**
  - **… mit „*gesundem Menschenverstand*“ und der „*Erfahrung*“ („*Intuition*“?) eines Schema- bzw. Datenbank-Designers**
  - **… oder durch die Abbildung (kanonische Transformation) eines ER-Schemas in ein relationales Schema**

Objektiv(er)e Kriterien für die *Fehlerfreiheit*, *Güte* oder *Eignung* eines relationalen Schemas? (→ *Qualität des Schemaentwurfs*)

***Nachvollziehbare* (und nachprüfbare) *Beurteilung*,**
ob zur Darstellung gegebener fachlicher Anforderungen ein relationales Schema *besser* ist als ein anderes?

Relationaler Schemaentwurf: Worum geht es?

- **Die Qualität / Eignung eines relationalen Schemas kann auf zwei Ebenen betrachtet werden**

  - **Logische (bzw. konzeptionelle) Ebene**
    - **Verständlichkeit und Interpretierbarkeit der Bedeutung des Schemas, d.h. der Relationstypen und ihrer Attribute (→ Semantik)**
    - **„Schnitt“ der Relationstypen (d.h. welche Attribute werden in einem Relationstyp zusammengefasst)**

    Ein hinsichtlich dieser Ebene „gutes“ Schema vermittelt ein klares Verständnis des Schemas und unterstützt/erleichtert die Formulierung korrekter Anfragen

  - **Implementierungsebene / Physische Ebene**
    - **Im Vergleich zur logischen Ebene andere Kriterien zur Bewertung eines Schemaentwurfs, z.B. Speicherungsform zur Unterstützung effizienter Zugriffe, effiziente Unterstützung bestimmter Anfrage- oder Änderungsoperationen etc.**

Der Fokus des „klassischen“ relationalen DB-Entwurfs liegt auf der logischen Ebene. Dieser Aspekt des DB-Entwurfs ist Inhalt dieses Kapitels.

Relationaler Schemaentwurf: Worum geht es?

- **Zentrales Ziel des relationalen Schemaentwurfs ist die Verhinderung sogenannter *Update-Anomalien***

- **Update-Anomalien entstehen durch**
  - **die *Vermischung mehrerer fachlicher Konzepte* in einem Relationstyp, wodurch in der Regel *Datenabhängigkeiten* entstehen**
  - ***Datenredundanzen***

Was ist Datenredundanz ?

→ Die mehrfache Ablage / Speicherung desselben Fakts

Fakt: Zuordnung eines Werts zu einer Eigenschaft eines (individuellen) Objekts

Beispiel:

Objekt	Eigenschaft	Wert
	→ *Farbe*	*silber*

© AUDI AG

Relationaler Schemaentwurf: Worum geht es?

Es werden 3 Arten von Update-Anomalien unterschieden

- **Einfüge-Anomalie**
- **Änderungsanomalie**
- **Lösch-Anomalie**

Die Vermeidung solcher Anomalien ist Ziel der Theorie des relationalen Schema-Entwurfs. Sie betrachtet folgende Fragestellungen:

- **Wie kann man *Datenabhängigkeiten* beschreiben ?**

  ➡ **„funktionale Abhängigkeiten“**

- **Welche *Qualitätskriterien* für gibt es für ein relationales Schema?**

  ➡ **„Normalformen“**

- **Wie kann man ein „schlechtes“ relationales Schema (d.h. mit Update-Anomalien) in ein „gutes“ Schema (d.h. ohne Update-Anomalien) *umwandeln*?**

  ➡ **„Normalisierung“**

Update-Anomalien

Betrachten wir folgendes Beispielschema:

Buchbestand (<u>InvNr</u>, ISBN, Titel, Autoren)

Entleihe (<u>InvNr</u>, LeserNr, LeserName, Adresse, EntleihDatum)

Reservierung (<u>ISBN, LeserNr</u>, LeserName, Adresse, ResDatum)

Dieses Schema ermöglicht als Vorteil für jede Entleihe und Reservierung den direkten Zugriff auf die Leserdaten, hat aber einige Nachteile zur Folge:

- **<u>Einfüge-Anomalie:</u>**
  Die Erfassung eines neuen Lesers, der nicht sofort ein Buch ausleiht oder reserviert, ist nicht möglich oder hat Nullwerte in den übrigen Attributen von Entleihe bzw. Reservierung zur Folge.

- **<u>Änderungsanomalie:</u>**
  Ändert sich die Leseradresse, müssen in allen Entleihe- und Reservierung-Tupeln die Adressdaten des Lesers geändert werden, obwohl fachlich nur eine einzige Änderung erfolgt.

- **<u>Lösch-Anomalie:</u>**
  Ein Leser gibt alle entliehenen Bücher zurück. Bestehen für sie/ihn keine Reservierungen, werden damit auch die Leserdaten gelöscht und müssen bei der nächsten Ausleihe / Reservierung neu erfasst werden.

Funktionale Abhängigkeiten

Einfügeanomalie → ***Daten nicht (ohne weiteres) einfügbar …***
… aufgrund Vermischung verschiedener fachlicher Daten

Änderungsanomalie → ***mehrfache Datenänderung nötig …***
… aufgrund Vermischung

Lösch-Anomalie → ***ungewollte Löschung von Daten …***
… aufgrund Vermischung

Die Vermischung fachlicher Konzepte hat offensichtlich Abhängigkeiten zwischen Attributen (und damit den Daten) zur Folge.
Sie ist wesentliche Ursache für die Update-Anomalien

Wie kann man solche Abhängigkeiten beschreiben, um darauf aufbauend Qualitätskriterien für Relationstypen zu definieren?

Zentrales „Werkzeug“ hierfür ist die funktionale Abhängigkeit …

Funktionale Abhängigkeiten

- **Eine funktionale Abhängigkeit gibt eine Abhängigkeit zwischen Mengen von Attributen (und damit zwischen Daten!) an und ist wie folgt definiert:**

  - **Gegeben ein Relationstyp R mit Attributemenge $\{A_1, \ldots, A_n\}$ und zwei Teilmengen X und Y von $\{A_1, \ldots, A_n\}$.**

  - **Eine funktionale Abhängigkeit (FD) zu R zwischen den Teilmengen X und Y von $\{A_1, \ldots, A_n\}$ gibt folgende <u>Werteabhängigkeit</u> (Einschränkung) für die Tupel einer beliebigen Relation r zu R an:**

    „Wenn zwei Tupel t1, t2 $\in$ r auf X gleiche Werte haben, dann müssen sie auch auf Y gleiche Werte haben."

    Als logischer Ausdruck formuliert:

    $$(\forall\ t1, t2 \in r)\ (\ t1[\ X\ ] = t2\ [\ X\ ] \Rightarrow t1[\ Y\ ] = t2\ [\ Y\ ]\ )$$

  - **Schreibweise für FD: $X \rightarrow Y$**

    Eine FD kann zur Vereinfachung auch ohne Mengenklammern geschrieben werden (d.h. $\{B_1, B_2, \ldots\} \rightarrow \{C_1, C_2, \ldots\}$ und $B_1, B_2, \ldots \rightarrow C_1, C_2, \ldots$ bedeuten dasselbe).

Funktionale Abhängigkeiten
Beispiel

Beispiel:

Relationstyp **Entleihe (InvNr, LeserNr, LeserName, Adresse, EntleihDatum)**

FDs zu Entleihe: **1) {InvNr} → {LeserNr, EntleihDatum}**
2) {LeserNr} → {LeserName, Adresse}

Relation zu Entleihe:

1)

InvNr	LeserNr	LeserName	Adresse	EntleihDatum
019-2375	100589	Gruber	Heidelberg, …	25.04.2010
019-1672	100951	Ackermann	Mannheim, ..	27.07.2010
012-9821	100589	Gruber	Heidelberg, …	17.05.2010
700-6238	100589	Gruber	Heidelberg, …	17.05.2010
701-6205	100951	Ackermann	Mannheim, …	30.07.2010
400-7208	350554	Haber	Weinheim,…	01.08.2010
300-3411	290432	Müller	Speyer, …	15.02.2010

2)

- Da InvNr der Schlüssel von Entleihe ist, können in einer zulässigen (d.h. die Schlüsselbedingung erfüllenden) Relation nie zwei Tupel mit gleichem Wert auf InvNr existieren.
  Die durch FD 1) angegebene Einschränkung ist somit immer erfüllt.

- Die obige Relation erfüllt auch die durch FD 2) angegebene Einschränkung:
  Für die beiden Werte von LeserNr, die mehrfach vorkommen, sind LeserName und Adresse gleich

Funktionale Abhängigkeiten

- **Die durch eine FD X → Y zu einem Relationstyp R angegebene Einschränkung besagt:**
  Haben zwei Tupel in einer Relation zu R gleiche Werte(kombinationen) auf X, dann müssen auch die Werte(kombinationen) auf Y gleich sein.

  → Die Werte auf Y *hängen* von den Werten auf X *ab*;
  die Werte für X *bestimmen* die Werte für Y

  Sprechweise daher: „Y ist funktional abhängig von X“,
  oder auch „X bzw. die Werte zu X bestimmen gemeinsam Y bzw. die Werte zu Y“

- **Die Angabe von einer oder mehreren FDs zu einem Relationstyp R schränkt ein, welche Tupel in einer Relation zu R vorkommen dürfen.**

  ➡ **eine FD ist eine Integritätsbedingung !**

- **Wie jede IB schränkt also auch eine FD die Menge der <u>möglichen</u> Relationen ein auf diejenigen, in denen die FD <u>erfüllt</u> ist (d.h. die FD <u>gültig</u> ist)**

Funktionale Abhängigkeiten

- **Ein Schlüssel kann als FD betrachtet werden, d.h. Schlüssel sind eine spezielle Form von FDs … :**

  Die Aussage „S ist Schlüssel zu Relationstyp R mit Attributemenge $\{A_1, \ldots, A_n\}$“ entspricht der FD $S \rightarrow \{A_1, \ldots, A_n\}$

  Beispiel:
  „InvNr ist Schlüssel zu Entleihe“ entspricht der FD
  {InvNr} → {InvNr, LeserNr, LeserName, Adresse, EntleihDatum}

- **Zu jedem Relationstyp R gibt es eine Reihe von trivialen, offensichtlichen FDs, die immer erfüllt sind, z.B.:**
  - **Für jedes Attribut A von R gilt: {A} → {A}**
  - **Für alle Teilmengen X und Y der Attribute zu R mit $Y \subseteq X$ gilt: X → Y**

InvNr	LeserNr	LeserName	Adresse	EntleihDatum
019-2375	100589	Gruber	Heidelberg, …	25.04.2010
019-1672	100951	Ackermann	Mannheim, ..	27.07.2010
012-9821	100589	Gruber	Heidelberg, …	17.05.2010
700-6238	100589	Gruber	Heidelberg, …	17.05.2010
701-6205	100951	Ackermann	Mannheim, …	30.07.2010
400-7208	350554	Haber	Weinheim,…	01.08.2010
300-3411	290432	Müller	Speyer, …	15.02.2010

→ Im folgenden werden nur nicht-triviale FDs betrachtet !

Funktionale Abhängigkeiten und Normalformen

- Basierend auf FDs und Schlüsseln wurden von Codd zunächst 3 Normalformen für Relationstypen vorgeschlagen („1. – 3. Normalform“)
- Ziel der Normalformen ist es, bestimmte Arten von Datenabhängigkeiten zu vermeiden, um
  - die Vermischung fachlicher Konzepte in einem Relationstyp zu verhindern
  - Datenredundanzen in Relationen zu verhindern

➡ Ziel: Verhindern von Update-Anomalien (vgl. Entleih-Relation aus Beispiel)

- Die Normalformen definieren Qualitäts-/Gütekriterien für „gut“ entworfene Relationstypen

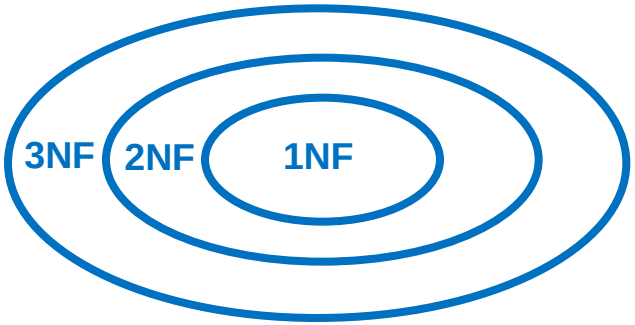
  „Gut“ heisst: Bestimmte Datenredundanzen (und damit Update-Anomalien) können in einer Relation des Relationstyps nicht auftreten.

- „Normalisierung“ eines Relationstyps R:

  Überführung von R in einen oder mehrere Relationstypen, die gut entworfen sind, d.h. Normalformen erfüllen

Normalformen

Die drei Normalformen bauen aufeinander auf.
Sie stellen immer schärfer werdende Kriterien bzw. Bedingungen dar:



- **Ist ein Relationstyp in dritter Normalform (3NF),**
  so ist er auch in zweiter und erster Normalform (2NF, 1NF)
- **Ist ein Relationstyp in 2NF, so ist er auch in 1NF**
- **Die Umkehrungen gelten nicht !**

Erste Normalform (1NF)

- Ein Relationstyp ist in erster Normalform, wenn alle seine Attribute nur atomare Wertebereiche haben
  - Zur Erinnerung:
    Atomar bedeutet: Die Werte des Wertebereichs sind nicht weiter zerlegbar (sind „einfach“)
    Als Werte eines Attributs sind somit keine Mengen oder zusammengesetzten Werte erlaubt
- Heutzutage ist die 1NF Teil der Definition eines Relationstyps! (siehe Kap.4)

Beispiele:

Artikel

ArtikelNr	…	AlternativeArtikel
4711	…	{4723, 5219}
4723	…	{4711, 5219}
4816	…	{ }
5219	…	{4711, 4723}
6529	…	{7013, 7093, 8093}
…	…	…

„keine Wiederholgruppen“

Kunde

…	Adresse Ort, PLZ, Str, Hausnr
…	(Köln, 50790, Deutzring, 41)
…	(Ingolstadt, 85049, Hauptstr., 18)
…	…

„keine strukturierten Werte“

Buch („Variante“)

InvNr	Autoren	…
270-2205	{Date, Darwen}	…
270-1156	{Miller, Johns, Tyler }	…
245-9032	{Jarrod}	…
…	…	…

Normalformen
Volle funktionale Abhängigkeiten

Die zweite Normalform baut auf dem Begriff einer „vollen“ FD auf.

Volle funktionale Abhängigkeit

- **Eine funktionale Abhängigkeit X → Y ist eine volle funktionale Abhängigkeit, wenn die Entfernung eines (beliebigen!) Attributs A aus X zur Folge hat, dass die funktionale Abhängigkeit nicht mehr gilt.**
  Sprechweise: Y ist voll funktional abhängig von X

  Dies bedeutet: Y ist funktional abhängig von X,
  aber für jedes A ∈ X ist Y nicht funktional abhängig von X \ {A}
  (d.h. die FD X \ {A} → Y gilt nicht).

- **Eine funktionale Abhängigkeit ist eine partielle funktionale Abhängigkeit, wenn aus X ein Attribut entfernt werden kann, die (dadurch entstehende) funktionale Abhängigkeit aber gilt.**

Beispiel:
Relationstyp Entleihe (InvNr, LeserNr, LeserName, Adresse, EntleihDatum)

{InvNr, LeserNr} → {EntleihDatum} ist eine partielle FD

Grund: LeserNr kann entfernt werden, d.h. wenn {InvNr, LeserNr} → {EntleihDatum} gilt, gilt auch {InvNr} → {EntleihDatum}
(Gilt dies auch für die Entfernung von InvNr ?)

Normalformen
Zweite Normalform

Schlüsselattribut: Attribut, dass in (mindestens) einem Schlüssel von R vorkommt

Nicht-Schlüsselattribut: Attribut, dass in keinem Schlüssel von R vorkommt.

Zweite Normalform (2NF)

Ein Relationstyp R ist in **zweiter Normalform**, wenn gilt:

(1) R ist in 1NF und

(2) jedes Nicht-Schlüsselattribut von R ist voll funktional abhängig vom Schlüssel* von R

Erläuterungen:

- Ziel der 2NF: Erkennen/Verhindern partieller Abhängigkeiten von einem Schlüssel („keine Abhängigkeit eines Nicht-Schlüsselattributs von einer (echten) Teilmenge des Schlüssels“)
- Eine Verletzung der 2NF ist offensichtlich nur möglich, wenn es einen Schlüssel mit mindestens 2 Attributen gibt (zusammengesetzter Schlüssel)
- Zusammengesetzte Schlüssel treten u.a. bei der Darstellung von Beziehungen (z.B. Kunde ←→ Produkt) oder komplex strukturierter Objekte (z.B. Bestellung mit Positionen) auf
- Die 2NF „prüft“ bezogen auf Schlüssel, ob mehrere fachliche Konzepte in einem Relationstyp vermischt sind (z.B. (redundante) Aufnahme von Produktinformationen in den Relationstyp, der die Beziehung zwischen Kunde und Produkt abbildet)

* Oft hat ein Relationstyp nur einen Schlüssel. Besitzt ein Relationstyp mehrere Schlüssel, so ist die 2NF erfüllt, wenn Bedingung (2) für alle Schlüssel gilt

Normalformen
Zweite Normalform: Beispiele

Beispiel:

Relationstypen Produkt, Kunde und InteressiertSichFür:

Schlüssel

ProduktNr	Produktbez.	Gewicht	…
6587	Drehstuhl	…	…
…	…	…	…

Schlüssel

KundenNr	Name	…
87006	Müller	…
89120	Meier	…

Schlüssel

ProduktNr	*KundenNr*	Produktbez.	Anfragedatum
6587	87006	Drehstuhl	01.12.2010
6587	89120	Drehstuhl	06.12.2010
…	…	…	…

mit FD {ProduktNr} → {Produktbez.}

Relationstyp InteressiertSichFür nicht in 2NF

Beispiel:

Relationstyp Entleihe (InvNr, LeserNr, LeserName, Adresse, EntleihDatum)

mit FDs 1) {InvNr} → {LeserNr, EntleihDatum}
2) {LeserNr} → {LeserName, Adresse}

1)

InvNr	LeserNr	LeserName	Adresse	EntleihDatum
019-2375	100589	Gruber	Heidelberg, …	25.04.2010
019-1672	100951	Ackermann	Mannheim, ..	27.07.2010
012-9821	100589	Gruber	Heidelberg, …	17.05.2010
700-6238	100589	Gruber	Heidelberg, …	17.05.2010
701-6205	100951	Ackermann	Mannheim, …	30.07.2010
400-7208	350554	Haber	Weinheim,…	01.08.2010
300-3411	290432	Müller	Speyer, …	15.02.2010

2)

Offensichtlich in 2NF, da es nur einen Schlüssel gibt und dieser aus nur einem Attribut besteht.

Erkenntnis hieraus:
2NF erfüllt, dennoch unerwünschte Effekte vorhanden
→ 2NF verhindert nicht alle Anomalien → weiterführende Normalform nötig

Normalformen
Transitive Abhängigkeit

Die dritte Normalform baut auf dem Begriff einer „transitiven“ Abhängigkeit auf.

Transitive Abhängigkeit

Gegeben ein Relationstyp R mit Attributemenge $\{A_1, \ldots, A_n\}$ und $X,Y \subseteq \{A_1, \ldots, A_n\}$.
Y ist transitiv abhängig von X, wenn es eine Attributemenge $Z \subseteq \{A_1, \ldots, A_n\}$ gibt mit folgenden Eigenschaften:

a) **Es gelten die FDs $X \rightarrow Z$ und $Z \rightarrow Y$**
b) **Die FD $Z \rightarrow X$ gilt nicht**

D.h. es gilt die transitive Abhängigkeit $X \rightarrow Y$ (= Eigenschaft a)

und die transitive Abhängigkeit ist „echt“ und nicht trivial (= Eigenschaft b)
(insbesondere kann Z damit kein Schlüssel sein, da dann $Z \rightarrow X$ für jede Menge X gelten würde)

Beispiel (Relationstyp Entleihe, siehe vorher):

{InvNr} → {LeserNr} und {LeserNr} → {LeserName, Adresse} sind FDs zu Entleihe
Die FD {LeserNr} → {InvNr} gilt offensichtlich nicht.
Damit ist {LeserName, Adresse} transitiv abhängig von {InvNr};
anders formuliert: Es gilt die transitive Abhängigkeit {InvNr} → {LeserName, Adresse}

Dritte Normalform (3NF)

Gegeben ein Relationstyp R mit Attributemenge $\{A_1, \ldots, A_n\}$, sowie $X \subseteq \{A_1, \ldots, A_n\}$ und $A \in \{A_1, \ldots, A_n\}$.

R ist in dritter Normalform, wenn gilt:

(1) R ist in 1NF und

(2) für jede FD $X \rightarrow \{A\}$ zu R mit $A \notin X$ gilt:

a) X beinhaltet einen Schlüssel von R (bzw. X ist selbst ein Schlüssel von R) oder

b) A ist ein Schlüsselattribut

Erläuterungen :

- **Ziel der 3NF: Erkennen/Verhindern transitiver (=indirekter) Abhängigkeiten von einem Schlüssel („alle Nichtschlüsselattribute hängen direkt von einem Schlüssel ab“)**
- **Die 3NF verbietet „echte“ transitive Abhängigkeiten (d.h. solche, bei denen die Menge Z kein Schlüssel ist)**
  - **Eine solche transitive Abhängigkeit deutet auf eine Vermischung von nicht zusammengehöriger fachlicher Informationen in einem Relationstyp hin (siehe Anfang des Kapitels)**
  - **Die 3NF „prüft“ somit, ob eine solche Vermischung vorliegt.**
- **Eine Verletzung der 3NF bedeutet, dass die Bedingungen (2) a) und (2) b) nicht erfüllt sind**
  - **Verletzung von a) bedeutet: X ist keine Obermenge eines Schlüssels. X kann somit höchstens eine Teilmenge eines Schlüssels sein**
  - **Verletzung von b) bedeutet: A ist ein Nichtschlüsselattribut**

In einem solchen Fall besteht eine echte transitive Abhängigkeit oder es ist bereits die 2NF verletzt.

Normalformen
Dritte Normalform: Beispiel

Fortführung des vorherigen Beispiels:

Relationstyp Entleihe (InvNr, LeserNr, LeserName, Adresse, EntleihDatum)

mit FDs 1) {InvNr} → {LeserNr, EntleihDatum}
2) {LeserNr} → {LeserName, Adresse}

1)

InvNr	LeserNr	LeserName	Adresse	EntleihDatum
019-2375	100589	Gruber	Heidelberg, …	25.04.2010
019-1672	100951	Ackermann	Mannheim, ..	27.07.2010
012-9821	100589	Gruber	Heidelberg, …	17.05.2010
700-6238	100589	Gruber	Heidelberg, …	17.05.2010
701-6205	100951	Ackermann	Mannheim, …	30.07.2010
400-7208	350554	Haber	Weinheim,…	01.08.2010
300-3411	290432	Müller	Speyer, …	15.02.2010

2)

Ist in 2NF (siehe vorher),
dennoch unerwünschte Effekte vorhanden!
→ ist 3NF verletzt?

Um zu zeigen, dass Entleihe nicht in 3NF ist, genügt es, eine Konstellation zu finden, die die Bedingungen der 3NF verletzt !

- **Einziger Schlüssel von Entleihe ist {InvNr}.**
- **Aus der FD {LeserNr} → {LeserName, Adresse} folgt, dass offensichtlich auch die FD {LeserNr} → {LeserName} gilt.**

{LeserNr} → {LeserName} und es gilt A ∉ X → **Verletzung der 3NF!**

X: Beinhaltet keinen Schlüssel von Entleihe

A: Kein Schlüsselattribut

Entleihe ist nicht in 3NF

Normalisierung

- **Mit den Normalformen ist ein (prüfbares!) „Qualitätsmaß“ für Relationstypen (und damit für ein relationales DB-Schema) gegeben**
  - **Die Normalformen ermöglichen die Erkennung partieller (2NF) und transitiver (3NF) Datenabhängigkeiten (und damit die Erkennung der Vermischung verschiedener fachlicher Sachverhalte)**
  - **Die 3NF stellt sicher, dass solche Vermischung nicht auftritt, wodurch die beschriebenen Dateninkonsistenzen und Update-Anomalien verhindert werden.**

Ziel somit: Alle Relationstypen eines relationalen DB-Schemas in 3NF

- **Dies wird erreicht durch „Normalisierung“**

Vorgehensweise (Verfahren), die sicherstellt, dass die Relationstypen eines Schemas in 3NF sind

Normalisierung

Relationales DB-Schema S
mit Relationstypen

Normalisierung durch Zerlegung

Normalisiertes, „gleichwertiges“ relationales DB-Schema S‘,
ggfs. mit neuen / geänderten Relationstypen, alle in 3NF!

- **Normalisierung:**
  Überführung eines beliebigen DB-Schemas in ein Schema, das die 3NF erfüllt
  (Hinweis: „Schema in 3NF“ = alle Relationstypen in 3NF)

- **Idee der Vorgehensweise:**
  Zerlege einen Relationstyp, der nicht in 3NF ist, in mehrere Relationstypen, die die vermischten fachlichen Sachverhalte trennen.
  Dadurch können die verletzenden (transitiven) FDs in den neuen Relationstypen nicht mehr auftreten.

Normalisierung
Vorgehensweise (Algorithmus)

Gegeben: Relationstyp R in 1NF* mit Schlüssel(n) und FDs
Ziel: Überführung des Relationstyps in 3NF durch Normalisierung

Führe hierzu folgende Schritte aus:

Herstellung 2NF

1) Prüfe für jeden Schlüssel von R, der aus mehr als einem Attribut besteht, ob es Nichtschlüsselattribute gibt, die nur von einem ***Teil*** des Schlüssels funktional abhängig sind.
   Falls es solche Attribute gibt:
   - Bilde aus dem Teil des Schlüssels und den abhängigen Attributen einen neuen Relationstyp R‘. Der Teil des Schlüssels wird (Primär-)Schlüssel für R‘.
   - Entferne die abhängigen Attribute aus R
   - Der Teil des Schlüssels wird ein Fremdschlüssel in R (der auf R‘ verweist)

Herstellung 3NF

2) Prüfe auf Attribute, die nur indirekt (transitiv) von einem Schlüssel abhängen. Suche hierzu nach Nichtschlüsselattributen A von R, die von einer Attributemenge X funktional abhängig sind. Dabei muß X (mindestens) ein Nichtschlüsselattribut enthalten, darf aber keinen Schlüssel enthalten.
   Falls es solche indirekt abhängigen Attribute A gibt, gehe analog zu 1) vor:
   - Bilde aus X und den indirekt abhängigen Attributen einen neuen Relationstyp R‘. X wird (Primär-)Schlüssel für R‘.
   - Entferne die indirekt abhängigen Attribute aus R
   - X wird Fremdschlüssel in R (der auf R‘ verweist)

Hinweis:
Die Normalisierung eines relationalen Schemas erfolgt, indem Schritte 1) und 2) auf jeden Relationstyp des Schemas angewendet werden

Normalisierung
Beispiel

Beispielschema:

Buchbestand (InvNr, ISBN, Titel, Autoren)
mit FDs {InvNr} → {ISBN} und {ISBN} → {Titel, Autoren}

Entleihe (InvNr, LeserNr, LeserName, Adresse, EntleihDatum)
mit FDs {InvNr} → {LeserNr, EntleihDatum}
und {LeserNr} → {LeserName, Adresse}

Reservierung (ISBN, LeserNr, LeserName, Adresse, ResDatum)
mit FDs {ISBN, LeserNr} → {ResDatum}
und {LeserNr} → {LeserName, Adresse}

(i) Relationstyp *Buchbestand*:

- **Schritt 1): Der einzige Schlüssel hat nur ein Attribut, also nichts zu tun**

- **Schritt 2): Nichtschlüsselattribute: ISBN, Titel, Autoren**
  Titel und Autoren hängen direkt von ISBN ab und indirekt vom Schlüssel
  Bilde neuen Relationstyp *Buch (ISBN, Titel, Autoren)*
  Entferne die indirekt abhängigen Attribute aus Buchbestand: *Buchbestand' (InvNr, ISBN)*
  (ISBN ist Fremdschlüssel in Buchbestand')

Normalisierung
Beispiel

Beispielschema:

Buchbestand (InvNr, ISBN, Titel, Autoren)
mit FDs {InvNr} → {ISBN} und {ISBN} → {Titel, Autoren}

Entleihe (InvNr, LeserNr, LeserName, Adresse, EntleihDatum)
mit FDs {InvNr} → {LeserNr, EntleihDatum}
und {LeserNr} → {LeserName, Adresse}

Reservierung (ISBN, LeserNr, LeserName, Adresse, ResDatum)
mit FDs {ISBN, LeserNr} → {ResDatum}
und {LeserNr} → {LeserName, Adresse}

(ii) Relationstyp *Entleihe*:

- **Schritt 1): Der einzige Schlüssel hat nur ein Attribut, also nichts zu tun**

- **Schritt 2): Nichtschlüsselattribute: LeserNr, LeserName, Adresse, EntleihDatum**
  LeserName, Adresse hängen direkt von LeserNr ab und indirekt vom Schlüssel
  Bilde neuen Relationstyp *Leser (LeserNr, LeserName, Adresse)*
  Entferne die indirekt abhängigen Attribute aus Entleihe:
  Entleihe' (InvNr, LeserNr, EntleihDatum) **(mit LeserNr als Fremdschlüssel)**

Normalisierung
Beispiel

Beispielschema:

Buchbestand (InvNr, ISBN, Titel, Autoren)
mit FDs {InvNr} → {ISBN} und {ISBN} → {Titel, Autoren}

Entleihe (InvNr, LeserNr, LeserName, Adresse, EntleihDatum)
mit FDs {InvNr} → {LeserNr, EntleihDatum}
und {LeserNr} → {LeserName, Adresse}

Reservierung (ISBN, LeserNr, LeserName, Adresse, ResDatum)
mit FDs {ISBN, LeserNr} → {ResDatum}
und {LeserNr} → {LeserName, Adresse}

(iii) Relationstyp *Reservierung*:

- **Schritt 1): Schlüssel ist {ISBN, LeserNr}**
  LeserName und Adresse sind von LeserNr (Teilmenge des Schlüssels) abhängig.
  Der neu zu bildende Relationstyp Leser wurde bereits bei der Normalisierung von Entleihe erstellt.
  Entferne die abhängigen Attribute aus Reservierung:
  Reservierung' (ISBN, LeserNr, ResDatum) **(mit LeserNr als Fremdschlüssel)**

- **Schritt 2): Ist jetzt auf Reservierung' anzuwenden.**
  Das einzige Nichtschlüsselattribut ResDatum ist voll funktional abhängig vom Schlüssel, somit ist nichts zu tun

Normalisierung
Zusammenfassung des Beispiels

Beispielschema:

Buchbestand (InvNr, ISBN, Titel, Autoren)
mit FDs {InvNr} → {ISBN} und {ISBN} → {Titel, Autoren}

Entleihe (InvNr, LeserNr, LeserName, Adresse, EntleihDatum)
mit FDs {InvNr} → {LeserNr, EntleihDatum}
und {LeserNr} → {LeserName, Adresse}

Reservierung (ISBN, LeserNr, LeserName, Adresse, ResDatum)
mit FDs {ISBN, LeserNr} → {ResDatum}
und {LeserNr} → {LeserName, Adresse}

Ergebnis der Normalisierung des Schemas

Buchbestand' (InvNr, ISBN)

Buch (ISBN, Titel, Autoren)

Entleihe' (InvNr, LeserNr, EntleihDatum)

Leser (LeserNr, LeserName, Adresse)

Reservierung' (ISBN, LeserNr, ResDatum)

(+ jeweils die Fremdschlüsselbedingungen und die entsprechend zugeordneten FDs des Ausgangsschemas…)

Zusammenfassung & Anmerkungen

Relationaler Schemaentwurf: Zusammenfassung

- **Ein relationales Schema kann Vermischungen von fachlichen Sachverhalten (Konzepten) beinhalten, die *Datenabhängigkeiten*, (unkontrollierte) *Datenredundanzen* und *Update-Anomalien* zur Folge haben.**

- **Die *Normalformen* definieren unterschiedlich strenge und überprüfbare Qualitätskriterien für Relationstypen und relationale DB-Schemata.**

- **Ziel der Normalformen ist die *Beseitigung* partieller (2NF) und indirekter / transitiver (3NF) *Abhängigkeiten* von einem Schlüssel.**

- ***Jedes* relationale DB-Schema kann durch Normalisierung in ein *gleichwertiges* Schema in 3NF überführt werden.**

Zusammenfassung & Anmerkungen

Normalisierung in der Praxis

- **Datenredundanz ist heute weniger ein „Platzproblem“ – kritischer ist die damit verbundene Fehleranfälligkeit!**

- **Normalisierung beseitigt bzw. minimiert Datenredundanz.**
  Allerdings entstehen dabei zusätzliche Relationstypen
  (und somit erhöht sich auch die Zahl der Datenbanktabellen).

  Diese bzw. die zugehörigen Tabellen müssen bei (komplexen) Anfragen durch Join-Operationen verknüpft werden, was sich nachteilig auf die Antwortzeit bzw. den Verarbeitungsaufwand des DBMS auswirkt.

- **Im jeweiligen konkreten Anwendungsfall ist daher beim Schemaentwurf abzuwägen, wie weit normalisiert wird (→ gezielte „Denormalisierung“)**

  Wird nicht vollständig normalisiert, so sind entsprechende Maßnahmen zur Vermeidung von Dateninkonsistenzen zu treffen (ansonsten: unkontrollierte Datenredundanz)!

- **In speziellen Anwendungsgebieten mit hohen Datenvolumen (Data Warehouses, analytische Anwendungen) werden Schemata im Entwurf gezielt denormalisiert zur „Anfragebeschleunigung“**

Zusammenfassung & Anmerkungen

Anmerkungen

- **Die klare Trennung fachlicher Sachverhalte ist üblicherweise eines der Ziele des konzeptionellen DB-Entwurfs (Erstellung eines ER-Schemas oder UML-Klassendiagramms)**

- **Sind die dabei entstehenden Entitäts- und Beziehungstypen aus fachlicher Sicht sinnvoll gebildet („geschnitten“) worden, so ist das relationale Schema, das mittels kanonischer Transformation aus dem ER-Schema abgeleitet werden kann, in der Regel bereits in 3NF.**

- **Im Prinzip können FDs und die Normalformen auch für ER-Schemata definiert werden, um sicherzustellen, das das Schema diesen Qualitätskriterien genügt. Wird dann die kanonische Transformation genutzt, ist eine Normalisierung des relationalen Schema nicht nötig.**

Verständnis- / Lernfragen zu diesem Kapitel

- ***Was ist die Zielsetzung des relationalen Schemaentwurfs?***
- ***Welcher Zusammenhang besteht zwischen dem konzeptionellen und dem relationalen Schemaentwurf? Warum ist der relationale Schemaentwurf notwendig?***
- ***Was bedeutet Datenredundanz ?***
- ***Was sind Update-Anomalien (Definition und Beispiel) ?***
- ***Was ist eine funktionale Abhängigkeit (Definition und Beispiel) ?***
- ***Was ist eine partielle, eine volle, eine transitive FD?***
- ***Was ist ein Nichtschlüsselattribut?***
- ***Welcher Zusammenhang besteht zwischen Integritätsbedingungen, FDs und Schlüsseln?***
- ***Welche Normalformen gibt es? Wie sind sie definiert und was bedeuten sie anschaulich? Wir hängen sie zusammen?***
- ***Wozu dient Normalisierung? Was ist dabei die Grundidee? Wie wird vorgegangen?***
- ***Kann jedes relationale Schema in ein normalisiertes Schema in 3NF überführt werden?***

Inhaltsübersicht

1. Einleitung und thematische Annäherung
2. Grundlagen, Grundkonzepte und Architektur von Datenbanksystemen
3. Konzeptioneller (fachlicher) Datenbank-Entwurf und das Entity-Relationship-Modell
4. Das relationale Datenmodell
5. SQL
6. Relationaler Schemaentwurf
7. Datenbank-Transaktionen und Transaktionsverwaltung
8. Physische Datenbank-Organisation
9. Ausblick auf weiterführende DBS-bezogene Themen

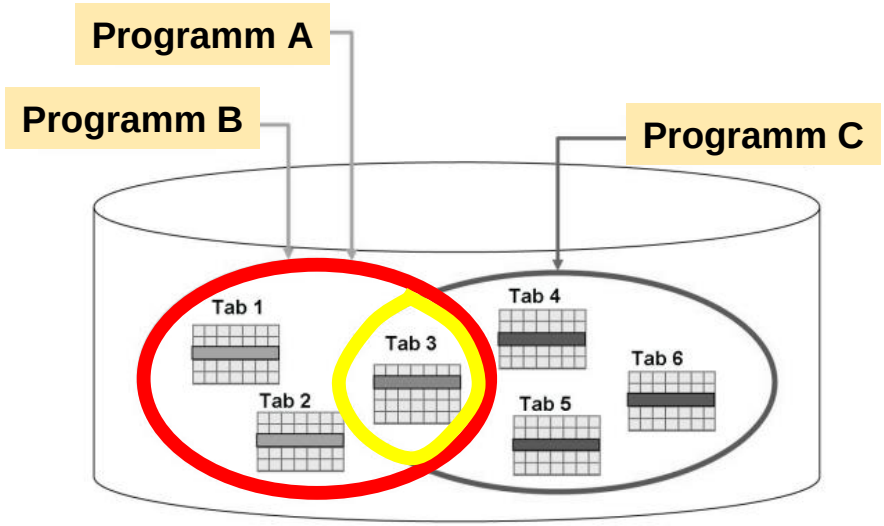
Transaktionen und Transaktionsverwaltung: Worum geht es?

Die Fähigkeit zum Mehrbenutzerbetrieb ist eine der grundlegenden Anforderungen an ein DBMS.

Dies bedeutet:

- ***Mehrere*** **Anwendungen bzw. Datenbank-Nutzer greifen …**
- **… *lesend und / oder schreibend* auf dieselbe DB zu …**
- **… *ohne gegenseitige Behinderung* oder falsche Zugriffs-/Anfrageergebnisse**

→ „nebenläufiger“ / „paralleler“ Zugriff auf bzw. Änderung von Daten in der DB



Beispiel:
Platzreservierung in einem Flugbuchungssystem

Transaktionen und Transaktionsverwaltung: Worum geht es?

- **Dabei sind nicht nur (wie bisher betrachtet) pro Anwender / Anwendung einzelne „atomare“ Operationen auf einer DB zu berücksichtigen, sondern auch mehrere, inhaltlich zusammengehörige Operationen auf einer DB.**

  Beispiele:

  - **Reservierungs-/Buchungssysteme (gemeinsame Reservierung von Flug und Hotel, …)**
  - **Wertpapierhandelssysteme (Ausbuchung und Einbuchung eines Wertpapierbestands zwischen Depots, …)**
  - **Warenwirtschaftssysteme (…), …**

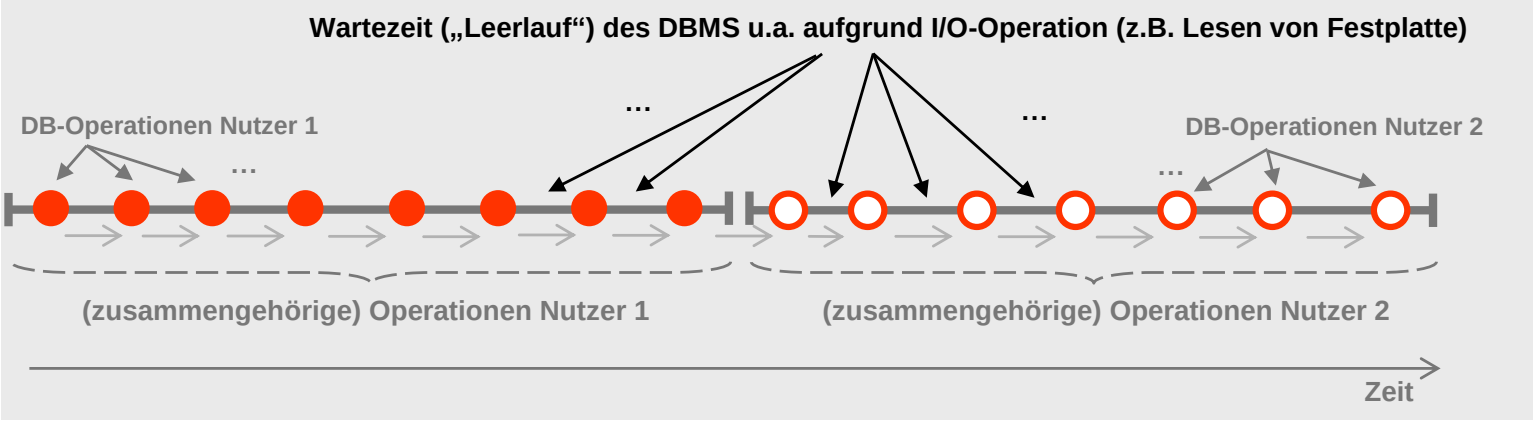
- **Ziel:**
  Das DBMS sollte sich idealerweise auch bei gleichzeitiger Nutzung durch mehrere Anwender für jeden einzelnen Anwender so verhalten, als ob er/sie es exklusiv (d.h. alleine) nutzen würde…

  - **Keine (bzw. möglichst geringe) *zeitliche Beeinträchtigung* der Arbeit mit dem DBMS durch andere Anwender (→ hoher „Durchsatz“ an Anfragen bzw. Operationen auf Daten)**
  - ***Konsistenz der Daten* auch bei mehrfachen („parallelen“) Zugriffen**

Transaktionen und Transaktionsverwaltung: Worum geht es?

- **Naheliegendes Vorgehen:**
  Jeder der mit dem DBMS arbeitenden Nutzer (Anwender bzw. Anwendung) erhält für seine Operationen „exklusiven“ Zugriff auf das DBMS und die DB

  ➡ ***Streng sequentielle Abarbeitung*** **der Operationen und Operationsabfolgen der einzelnen Nutzer**

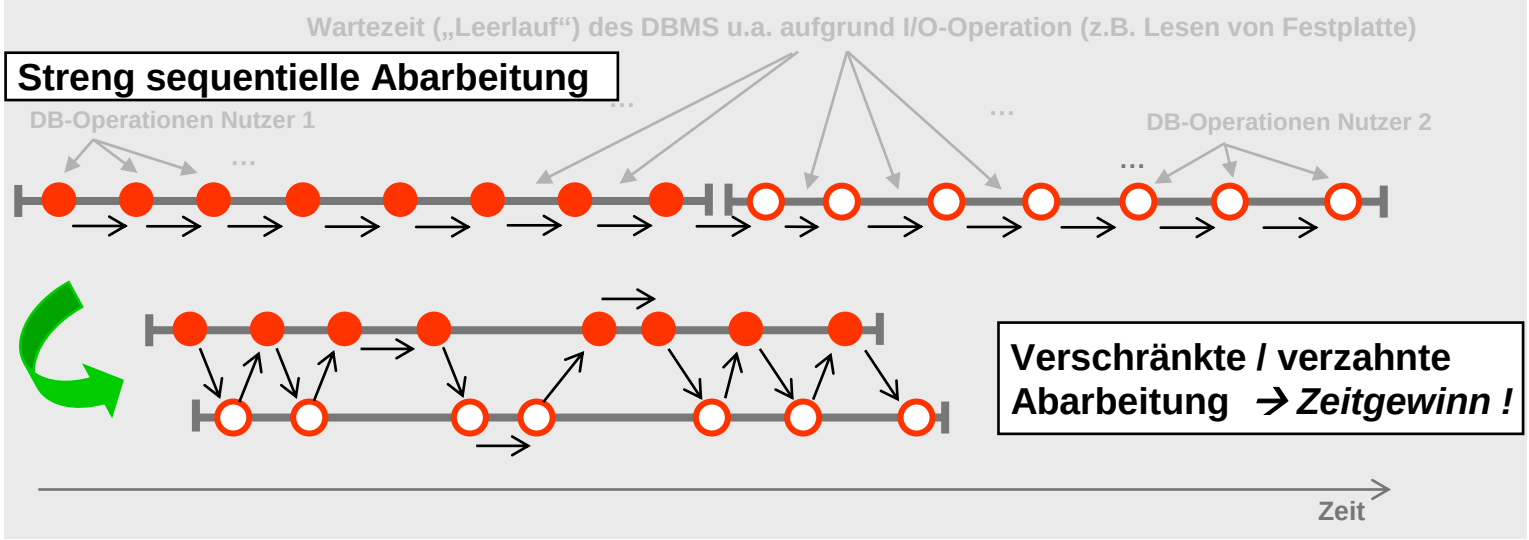


→ : Abarbeitungsreihenfolge

- ✓ **Keine gegenseitige Behinderung**
- ✓ **Keine Gefahr inkonsistenter Datenbestände durch konkurrierende Änderung**
- ⚡ ***Unnötige bzw. unnötig lange Wartezeiten*** **für einzelne Nutzer**
- ⚡ ***Verschwendung von Kapazitäten des DBMS*** **durch ungenügende Nutzung von Warte-/Verzögerungszeiten bei Zugriff auf Speichermedien (→ Verzögerungsfaktor 10^5 bis 10^6 !)**

Transaktionen und Transaktionsverwaltung: Worum geht es?

- **Um die nachteiligen Effekte zu vermeiden, ist es sinnvoll, die zusammengehörigen Operationen der gleichzeitig mit dem DBMS arbeitenden Nutzer soweit wie möglich zu „verschränken“:**

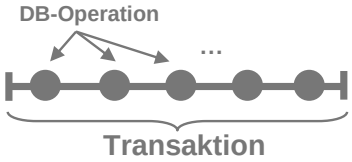


Was bedeutet „zusammengehörige Operationen“?
Wie weit können Operationen verschränkt werden?

Welche Konflikte können im Mehrbenutzerbetrieb zwischen gleichzeitig arbeitenden Nutzern (bzw. Anwendungen) auftreten? Und wie können sie verhindert werden?

Grundlegend für den Mehrbenutzerbetrieb und den Umgang mit Zugriffskonflikten sind zwei Konzepte bzw. Begriffe: Transaktion und Sperre

Transaktion (TA)



- **Eine TA ist eine Folge von Lese- oder Schreiboperationen auf einer DB, die eine logisch zusammengehörige Verarbeitungseinheit bilden**
  (Schreiboperation: Einfügen, Löschen, Ändern)

- **Die Ausführung von nur einzelnen Operationen der TA führt in der Regel zu einer inkonsistenten DB. Wird hingegen die TA <u>als Ganzes</u> (d.h. alle Operationen der TA) ausgeführt, so entsteht aus einer konsistenten DB durch Abarbeitung der TA wieder eine konsistente DB.**

- **Die Operationen (im System: SQL-Anweisungen) können in ein Anwendungsprogramm eingebettet sein oder interaktiv formuliert werden**
  (→ TAen sind für beide Fälle wichtig!)

Beispiel:

„Von Konto 4711 werden 1.000 Euro auf Konto 4814 überwiesen."

→ Transaktion bestehend aus 2 logischen Operationen
(Abbuchung auf Konto 4711, Zugang auf Konto 4814)

Im Detail betrachtet sind in der Regel noch mehr Operationen nötig !
(jeweils Lesen des Kontostands (SELECT) und Schreiben der Änderung (UPDATE))

Mögliche grundlegende Konflikte im Mehrbenutzerbetrieb

Mögliche Konflikte beim gemeinsamen („konkurrierenden“) Zugriff mehrerer TAen auf dieselben Daten einer DB

(im Zusammenhang mit TAen werden Daten oft auch als „Datenobjekt“ bezeichnet)

- **„Lost Update“**
- **„Dirty Read“**
- **„Unrepeatable Read“ / „Phantom Read“**

Zugrundegelegtes Beispiel für die folgenden Seiten:

- **Flugreservierungsdatenbank, pro Flug ein Datensatz, u.a. mit der Anzahl reservierter Sitzplätze**
- **X, Y, … bezeichnen im Folgenden einzelne Flug-Datensätze (bzw. das darin enthaltene Feld mit der Anzahl reservierter Plätze)**

Mögliche grundlegende Konflikte im Mehrbenutzerbetrieb: Lost Update

„Lost Update“ – Problem

Zeitliche Überschneidung der Operationen zweier TAen, die auf dasselbe Datenobjekt X zugreifen, wodurch die Änderung der ersten Transaktion verlorengeht.

Beispiel hier: Storno von 5 Platzreservierungen auf Flug (=TA1), Reservierung von 4 Plätzen für Flug (=TA2)

Zeit ↓

Wert für X in DB	TA1	Wert für X (aus Sicht TA1)	TA2	Wert für X (aus Sicht TA2)
80	*BOT (TA1)*			
	…		*BOT (TA2)*	
	read (X)	*80*	*…*	
	X := X – 5	*75*		
			read (X)	*80*
			X := X + 4	*84*
75	*write (X)*			
	EOT (TA1)			
84			*write (X)*	
			EOT (TA2)	

Statt des korrekten Werts 79 ergibt sich 84 als Endergebnis.
Die Änderung von TA1 (Storno von 5 Plätzen) ging „verloren“ (überschrieben durch TA2) !

Mögliche grundlegende Konflikte im Mehrbenutzerbetrieb: Dirty Read

„Dirty Read“ – Problem (andere Bezeichnung: „Temporary Update“-Problem)

Eine TA ändert ein Datenobjekt, muss dann aber vor Beendigung der TA abgebrochen werden*. Das geänderte Datenobjekt wird von einer anderen TA gelesen, bevor der ursprüngliche Wert wiederhergestellt wird

Beispiel hier: Storno von 5 Platzreservierungen auf Flug X (=TA1) muss rückgängig gemacht werden aufgrund logischer (z.B. Umbuchung nicht möglich) oder technischer Fehler (z.B. Fehler des Speichermediums)

Zeit ↓

Wert für X in DB	TA1	Wert für X (aus Sicht TA1)	TA2	Wert für X (aus Sicht TA2)
80	*BOT (TA1)*			
	…		*BOT (TA2)*	
	read (X)	*80*	*…*	
	X := X – 5	*75*		
75	*write (X)*			
			read (X)	*75*
			X := X + 4	*79*
79	*…*		*write (X)*	
			…	
…	*abort*			

TA2 hat „unsaubere“ Daten (dirty data) gelesen, d.h. Daten, die von einer noch nicht abgeschlossenen TA (TA1) erzeugt wurden, und arbeitet mit diesen weiter.

Mögliche grundlegende Konflikte im Mehrbenutzerbetrieb: Unrepeatable Read, Phantom Read

„Unrepeatable Read“ – Problem

Eine TA liest ein Datenobjekt zweimal. Zwischen den zwei Leseoperationen wird das Datenobjekt von einer anderen TA geändert, so dass die erste TA zwei unterschiedliche Werte erhält.

Beispiel:

- **Ein Kunde wünscht in einer Transaktion Auskunft zu verfügbaren Plätzen auf verschiedenen Flügen**
- **Entscheidet er/sie sich für einen Flug, wird vor Durchführung und Abschluss der Reservierung die Platzzahl nochmals gelesen und hat – abhängig von anderen noch in Abarbeitung befindlichen ändernden TAen – einen anderen Wert als zuvor**

➡ **Die lesende TA sieht einen Wert oder Datensatz, der beim zweiten Lesen nicht mehr rekonstruierbar ist oder nicht mehr existiert.**

Dieses Problem taucht in verschiedenen „Varianten“ auf, z.B. auch im Zusammenhang mit der Anwendung von Aggregatfunktionen (eine TA wendet z.B. die Summenfunktion an, während andere TAen Datenobjekte ändern, über die summiert wird) oder der Löschung / Einfügung von Datenobjekten, wodurch sich ebenfalls nicht reproduzierbare Werte ergeben können.

Dieser Effekt wird auch als „Phantom Read“-Problem bezeichnet.

Transaktionseigenschaften

Zur Vermeidung von Konflikten bzw. unerwünschten Effekten sind für TAen die sogenannten „ACID“-Eigenschaften wünschenswert:

Atomarität / Atomizität *(Atomicity)*

- **Eine TA ist eine atomare Verarbeitungseinheit:**
  Sie wird entweder ganz oder gar nicht ausgeführt.

Konsistenz *(Consistency)*

- **Eine TA bewahrt die Konsistenz: Wird sie auf einer konsistenten DB (vollständig) ausgeführt, so entsteht hierdurch wieder eine konsistente DB.**

Isolation *(Isolation)*

- **Eine TA ist isoliert von anderen TAen auszuführen:**
  Ihre Änderungen sind für andere Transaktionen solange nicht sichtbar, bis sie erfolgreich abgeschlossen wurde.
  (Andere Formulierung: Gleichzeitig ausgeführte TAen haben keine Auswirkungen auf das Ergebnis einer TA)

Dauerhaftigkeit *(Durability)*

- **Die von einer erfolgreich abgearbeiteten TA durchgeführten Änderungen werden dauerhaft in der DB festgeschrieben.**
  Sie gehen auch bei möglichen nachfolgenden Fehlern nicht verloren.

Transaktionseigenschaften
Beispiel

Beispiel:
„Von Konto 4711 werden 1.000 Euro auf Konto 4814 überwiesen.“
Transaktion bestehend aus 2 Operationen.

Mögliche Umsetzung der Operationen in SQL:

```
UPDATE Konto SET Kontostand = Kontostand - 1.000 WHERE Kontonr = 4711;
UPDATE Konto SET Kontostand = Kontostand + 1.000 WHERE Kontonr = 4814;
```

Atomarität / Atomizität

→ es werden beide Kontobuchungen (d.h. Operationen) ausgeführt oder keine

Konsistenz

→ da derselbe Betrag abgezogen bzw. addiert wird, ist die DB nach Ausführung der TA konsistent, wenn sie auch vor deren Ausführung konsistent war.
(Zwischen den beiden Operationen ist die DB in einem inkonsistenten Zustand!)

Isolation

→ die aktualisierten Kontostände sind erst nach erfolgreicher Ausführung der TA für andere TAen sichtbar (anders formuliert: findet z.B. parallel eine weitere Kontobewegung (TA) für Konto 4814 statt, so ist das Ergebnis dasselbe, als wenn beide TAen nacheinander stattgefunden hätten)

Dauerhaftigkeit

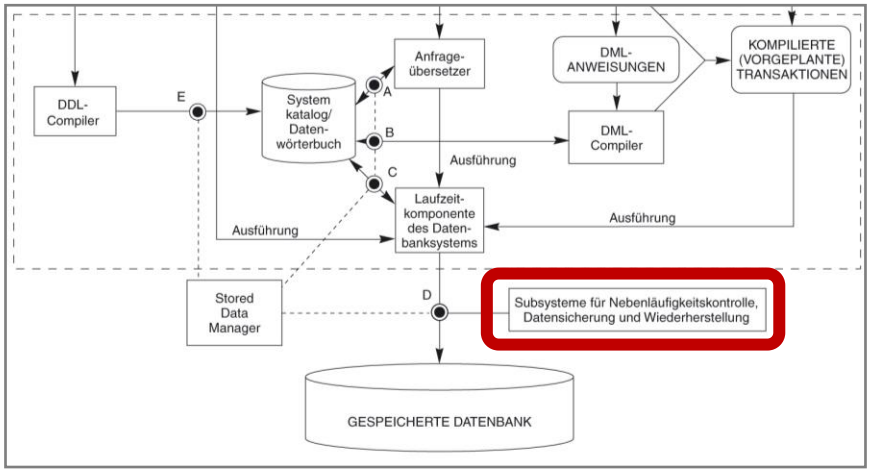
→ Nach der zweiten Operation werden für beide Konten die neuen Kontostände dauerhaft in die DB übernommen.

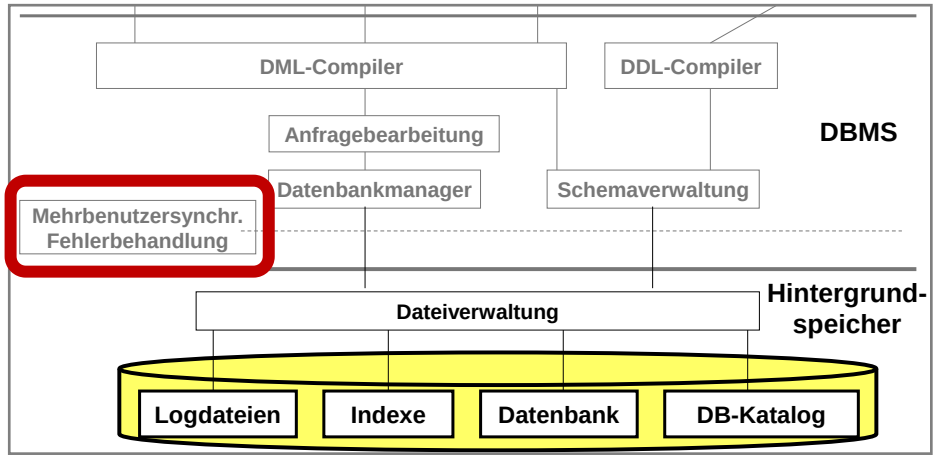
DBMS und Transaktionsverwaltung

Die Verwaltung und Synchronisierung von Transaktionen im Mehrbenutzerbetrieb erfolgt durch eine Systemkomponente („Transaktionsmanager“) des DBMS.

(genaugenommen: Mehrere zusammenspielende Komponenten …)

Erinnerung: Systemarchitektur DBS (vgl. Kap. 2)





Grundlegende Konzepte, die hierfür nötig sind: Sperren und Sperrprotokolle

(Diese Konzepte werden (neben anderen) zur Realisierung der Transaktionsverwaltung durch die Systemkomponente bzw. ihre Teilkomponenten implementiert)

Sperren und Sperrverwaltung

Vermeidung von Konflikten (und Zusicherung wünschenswerter TA-Eigenschaften) erfolgt durch „Abstimmung“ der Zugriffe mehrerer aktiver TAen auf dasselbe Datenobjekt.

- **Ziel:**
  „Gleichzeitige“ Abarbeitung (d.h. „parallel“ / verschränkt) möglichst vieler TAen, die sich nicht gegenseitig beeinflussen. Solche TAen heissen „serialisierbare TAen“

- **Zwei TAen sind serialisierbar, wenn die verschränkte Ausführung der TAen dasselbe Resultat liefert, wie wenn sie streng sequentiell ausgeführt worden wären (insbesondere bleibt somit die Reihenfolge der Operationen in jeder TA erhalten!).**

- **Nutzen:**
  - **Erhöhung des Durchsatzes an TAen (= Anzahl erfolgreich abgeschlossener TAen pro Zeiteinheit)**
  - **Kurze Transaktionen müssen nicht auf Beendigung langlaufender TAen warten**

Sperren und Sperrverwaltung

- **Mechanismus zur „Abstimmung“: Sperren („Locks“) von Datenobjekten**
  - **Einfachste (und in Praxis nicht akzeptable…) Lösung:**

    Sperre Tabelle vollständig für andere Anwender (TAen), solange ein Anwender (d.h. eine TA) Änderungen durchführt

    → Extremer, zu grober Ansatz, da nicht vertretbare Wartezeiten für andere Anwender

  - **Zur Sicherstellung der Datenkonsistenz genügt es, kollidierende Zugriffe auf den aktuell in Änderung befindlichen Datenobjekten zu verhindern**

- **Es werden zwei Sperrtypen (auch als Sperrmodi bezeichnet) unterschieden:**
  - **Lesesperre (Read Lock, Shared Lock, „SLOCK“)**

    → zeigt an, dass auf ein Datenobjekt ein lesender Zugriff erfolgt bzw. erfolgen soll

  - **Schreibsperre (Write Lock, Exclusive Lock, „XLOCK“)**

    → zeigt an, dass auf ein Datenobjekt ein schreibender (ändernder) Zugriff erfolgt bzw. erfolgen soll

Sperren und Sperrverwaltung

- **Nutzung / Einsatz der Sperren:**
  - **<u>Jede</u> TA fordert vor Durchführung einer Operation auf einem Datenobjekt eine Sperre für dieses Objekt an**
  - **Vor einer lesenden Operation: Anforderung einer Lesesperre**
  - **Vor einer schreibenden Operation: Anforderung einer Schreibsperre**
  - **Eine TA kann eine Operation nur durchführen, wenn sie zuvor für das Datenobjekt eine entsprechende Sperre erhalten hat.**
  - **Nach Durchführung der Operation: Freigabe (Löschen) der Sperre**

- **Gewährung und Verwaltung von Sperren durch eine Komponente des DBMS, den „Lock Manager“ (Teil des Transaktionsmanagers)**
  - **Lock Manager verwaltet eine Sperrtabelle (interne Systemtabelle des DBMS)**
  - **Grundprinzip:**
    - **Pro Sperre auf einem Datenobjekt ein Datensatz in der Sperrtabelle mit folgender Information: Name/ID des Datenobjekts, Sperrtyp, ID der sperrenden TA.**
    - **Zusätzlich Warteschlange mit den TAen, die ebenfalls auf dasselbe Datenobjekt zugreifen möchten.**

Sperren und Sperrverwaltung

- „Abstimmung“ zwischen konkurrierenden TAen:
  Ist ein Datenobjekt bereits durch eine TA1 gesperrt, so wird einer weiteren TA2 das Setzen einer Sperre auf dasselbe Datenobjekt (und damit der Zugriff auf das Datenobjekt) aufgrund folgender Sperrkompatibilitätstabelle erlaubt:

TA 2 / TA 1	Lesesperre	Schreibsperre
Lesesperre	Ja	Nein
Schreibsperre	Nein	Nein

- Für die Verwaltung von TAen durch das DBMS ist u.a. die unterstützte Sperrgranularität der Datenobjekte ein wichtiger Faktor
  - Sperrgranularität der Datenobjekte: „Größe“ der sperrbaren Datenobjekte

Feld → Datensatz → Block/Seite → Tabelle / Datei → Datenbank

fein → grob

  - Die Sperrgranularität der Datenobjekte bestimmt somit, wie grob oder fein Sperren gesetzt werden können.
  - feiner → hoher Aufwand für Sperrverwaltung;
    gröber → ggfs. unnötig lange Wartezeit für andere TAen!

Sperrprotokolle und Zwei-Phasen-Sperrprotokoll

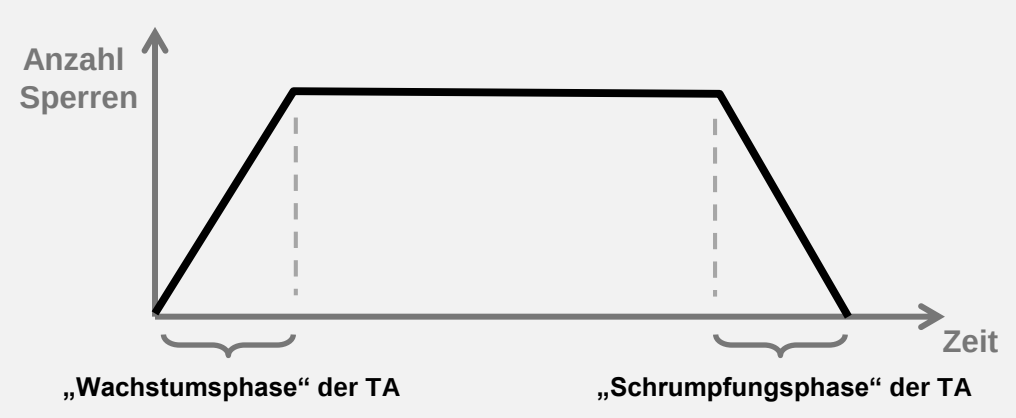
- **Problem: Oft konkurrieren viele TAen um dieselben Datenobjekte mit teilweise komplexen wechselseitigen Abhängigkeiten**

- **Sperren an sich garantieren noch nicht die Serialisierbarkeit („Verschränkbarkeit“) einer Menge von TAen.**

- **Aufbauend auf dem Prinzip der Sperren werden daher Sperrprotokolle definiert, die die Verschränkung (Serialisierbarkeit) von TAen gewährleisten**
  - **Protokoll: Menge von Regeln, denen die TA-Verarbeitung folgt**
  - **Sperrprotokoll ist implementiert im Transaktionsmanager**
  - **Bekanntestes und verbreitetes Sperrprotokoll: Zwei-Phasen-Sperrprotokoll**

Zwei-Phasen-Sperrprotokoll (Two Phase Lock Protocol - 2PL)

- **Eine TA gehorcht dem 2PL, wenn die TA <u>alle</u> ihre Sperren setzt, <u>bevor</u> die <u>erste Freigabe</u> einer Sperre durch die TA erfolgt.**



- **Nutzen: Gehorcht jede TA aus einer Menge von gemeinsam auszuführenden TAen dem 2PL, so ist die Menge der TAen garantiert (beweisbar…) serialisierbar**

Rücksetzen, Recovery und Backup – Ein Überblick

- **Eine Transaktion kann nach Ausführung von nur einigen ihrer Operationen durch das DBMS fehlschlagen.**

- **Mögliche Gründe hierfür:**
  - **Bei der TA-Abarbeitung erkannte Konsistenzverletzungen (z.B. zu geringer Kontostand)**
  - **Semantische (fachliche) Fehler im Anwendungsprogramm & Programmierfehler (Überlauf, Division durch Null, …)**
  - **Fehlersituation in DBMS-Software**
  - **Fehlersituation in Rechnersystem (Betriebssystem; Hardwarefehler, insbesondere Fehler eines Speichermediums)**
  - **Netzwerkfehler**
  - **Abbruch der TA durch Transaktionsmanager (z.B. aufgrund Deadlock-Situation)**
  - **DBMS wird während TA-Verarbeitung heruntergefahren**
  - **Ausnahme- / Katastrophensituationen (Ausfall Stromversorgung, Brand, Explosion, Erdbeben, …)**

Rücksetzen, Recovery und Backup – Ein Überblick

- **In einem solchen Fall ist das DBMS dafür verantwortlich, bereits durchgeführte Änderungen der TA in der DB *rückgängig* zu machen („*rücksetzen*“, engl. rollback)**

- **Um dies zu ermöglichen, führt das DBMS ein *Systemlog* (Log-Datei, „DBMS-Journal“), in dem die Operationen aller aktiven TAen protokolliert werden:**
  - **TA-Identifikator (eindeutige Nummer für Transaktion, von TA-Manager vergeben)**
  - **TA-Beginn**
  - **erfolgte Werteänderungen (alter Wert, neuer Wert)**
  - **usw. …**

- **Schlägt eine TA fehl, so nutzt der Recovery Manager (Systemkomponente des DBMS) das Log, um die Änderungen dieser TA rückgängig zu machen.**
  - **Gibt es bei langlaufenden TAen konsistente Zwischenzustände bei der TA-Abarbeitung, so muss nicht die komplette TA rückgängig gemacht werden.**
  - **Hierfür müssen sogen. Sicherungspunkte (savepoints) gesetzt werden, die der Recovery Manager gezielt wiederherstellen kann**

Rücksetzen, Recovery und Backup – Ein Überblick

- **Recovery (Wiederherstellung):**
  - **Aktivitäten zur Wiederherstellung eines vorherigen, konsistenten Datenbankzustands im Fehlerfall, basierend auf DBMS Log:**
    - **„Redo“-Operation:**
      Erneute Durchführung der Schritte derjenigen TAen, die zum Fehlerzeitpunkt bereits erfolgreich abgeschlossen, aber noch nicht persistiert waren (unter Verwendung des Logs).
    - **„Undo“-Operation:**
      Rücknahme der Schritte derjenigen TAen, die zum Fehlerzeitpunkt noch nicht erfolgreich abgeschlossen, aber ggfs. bereits persistiert waren (unter Verwendung des Logs).
  - **Hierzu zählen auch alle vorbeugenden Maßnahmen, die eine Wiederherstellung im Fehlerfall möglich machen, inkl. Datensicherung und Fortschreiben eines Systemlogs.**
- **Backup (Datensicherung):**
  - **Regelmäßige Sicherung der Daten der DB, um Datenverlust in Fehlersituationen zu vermeiden**
  - **Zu unterscheiden: Vollständiges und inkrementelles Backup**
  - **Achtung: „Backup“ ist nicht mit „Archivierung“ gleichzusetzen !**

Transaktionen in SQL

- **Bedeutung einer TA in SQL: Wie zuvor, d.h. „Ganz oder gar nicht“**
  - **Typischerweise wird jede einzelne SQL-Anweisung als eine TA ausgeführt.**
  - **Sollen mehrere SQL-Anweisungen als eine TA zusammengefasst werden, so sind hierfür explizite SQL-Anweisungen zur TA-Steuerung zu verwenden.**

- **SQL-Anweisungen zur TA-Steuerung:**
  - **START TRANSACTION**
  - **COMMIT**
    - **erfolgreiches Ende der TA, Übernahme aller Änderungen**
  - **ROLLBACK**
    - **Abbruch der TA mit Rücksetzen der DB auf den Stand vor Beginn der TA (Rücknahme aller bisherigen Änderungen der TA anhand des DBMS System Logs)**

- **Beispiel:**

```
START TRANSACTION;
  UPDATE Konto SET Kontostand = Kontostand -  1.000 WHERE Kontonr = 4711;
  UPDATE Konto SET Kontostand = Kontostand + 1.000 WHERE Kontonr = 4814;
COMMIT;
```

Transaktionen in SQL

- Angabe von Parametern für die Steuerung der Verarbeitung einer Transaktion in SQL mittels Anweisung `SET TRANSACTION`

- Wichtigste Optionen der Anweisung: Isolationsgrad und Zugriffsmodus

  - **Isolationsgrad** (Option `ISOLATION LEVEL`)
    - Gibt an, welche der beschriebenen grundlegenden TA-Konflikte in Kauf genommen werden
    - Mögliche Werte:
      - `READ UNCOMMITTED` (Dirty Read erlaubt)
      - `READ COMMITTED` (nur Daten abgeschlossener TAen sind lesbar)
      - `REPEATABLE READ` (es werden immer die Datenwerte gelesen, die bei Beginn der TA gültig waren)
      - `SERIALIZABLE` (Setzen von Lesesperre bei `SELECT`)
    - Lost Update (extrem kritisch!) ist generell verboten! (Schreibsperre)

  - **Zugriffsmodus** (Option `ACCESS MODE`)

    Lesezugriff oder Lese- & Schreibzugriff (Werte `READ ONLY`, `READ WRITE`)

Charakteristika relationaler Datenbanksysteme

Was ist der Fokus (die Stärke) von RDBSen ?

Die zentralen charakteristischen Merkmale bzw. Eigenschaften von RDBSen auf einen Blick: **(vgl. Kapitel 4 – 7)**

- **Verwaltung <u>strukturierter</u> Daten (→ schemabasiert, Relationen bzw. Tabellen)**
- **Unterstützung von lesenden <u>und schreibenden</u> Datenzugriffen im Mehrbenutzerbetrieb (→ Transaktionsverwaltung)**
- **Auf detailliertester Ebene letztlich „Einzelsatzverarbeitung“**

→ wichtige Stärken für klassische, operative Geschäftsanwendungen !

Für Anwendungen mit anderen „Anforderungsprofilen“ (z.B. ohne Schreibzugriffe) ergeben sich hieraus jedoch Beschränkungen … (siehe Kap. 9).

Verständnis- / Lernfragen zu diesem Kapitel

- ***Welche Anforderung an ein DBMS macht eine Transaktionsverwaltung notwendig?***
- ***Was bedeutet "Mehrbenutzerbetrieb" im Zusammenhang mit DBMS/DB?***
- ***Welche Konflikte können im Mehrbenutzerbetrieb auftreten? Erläutern Sie diese anhand geeigneter Beispiele.***
- ***Was ist eine Transaktion?***
- ***Welche Eigenschaften sollte eine Transaktion besitzen? Was bedeuten die einzelnen Eigenschaften?***
- ***Welche Bedeutung haben Sperren im Zusammenhang mit Transaktionen? Welche Sperrtypen gibt es?***
- ***Was ist Sperrgranularität?***
- ***Welche Vor-und Nachteile sind bei der Festlegung einer konkreten Sperrgranularität für ein DBMS zu berücksichtigen?***
- ***Was ist ein Sperrprotokoll? Was ist das Zwei-Phasen-Sperrprotokoll?***
- ***Wie werden Transaktionen in SQL abgebildet?***